



Client-Server: Model of Interaction



Reti di Calcolatori (Modulo 2)

Introduction

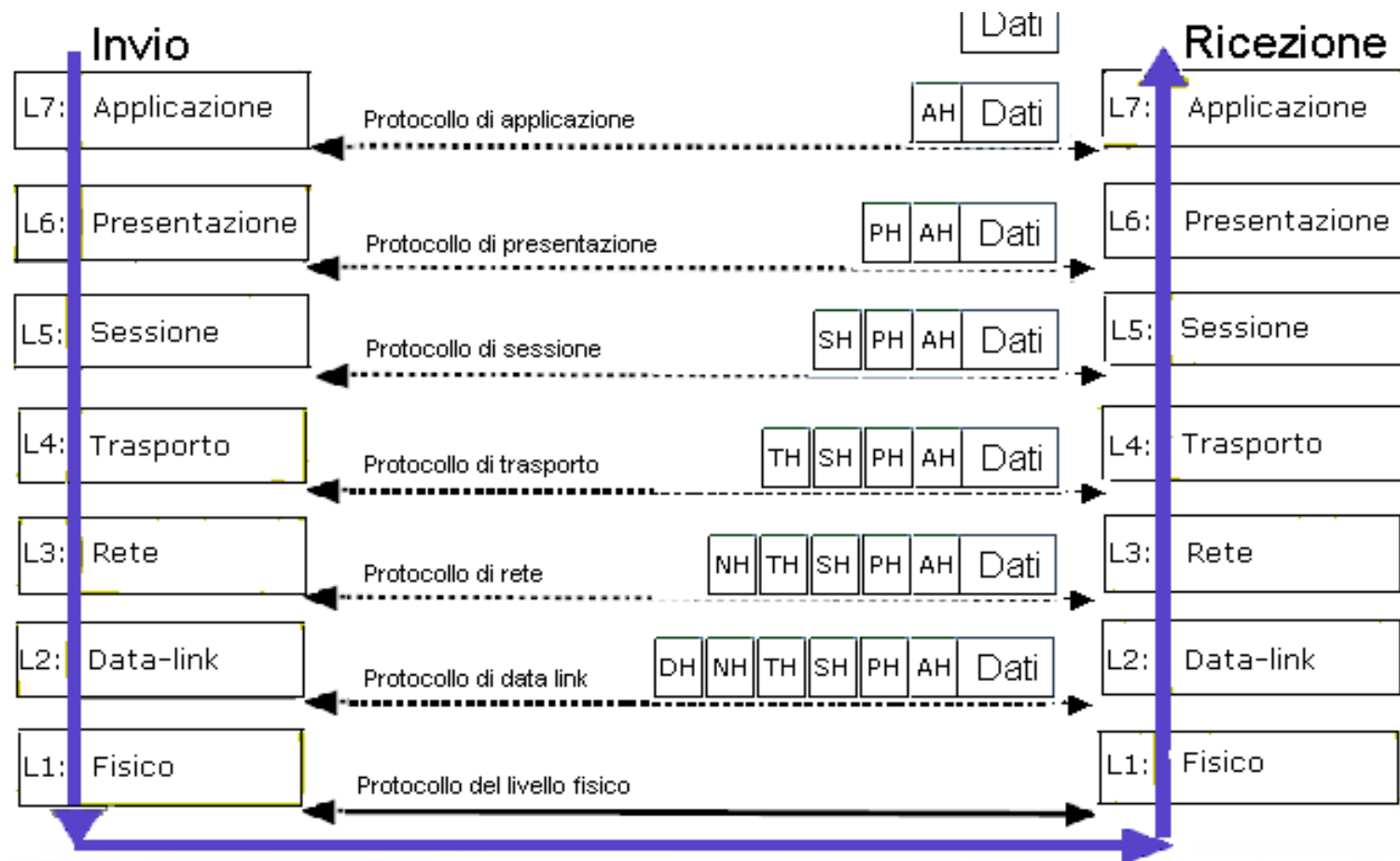
- ▶ Module I

- ▶ TCP/IP details and fundamental protocols
- ▶ Router architecture

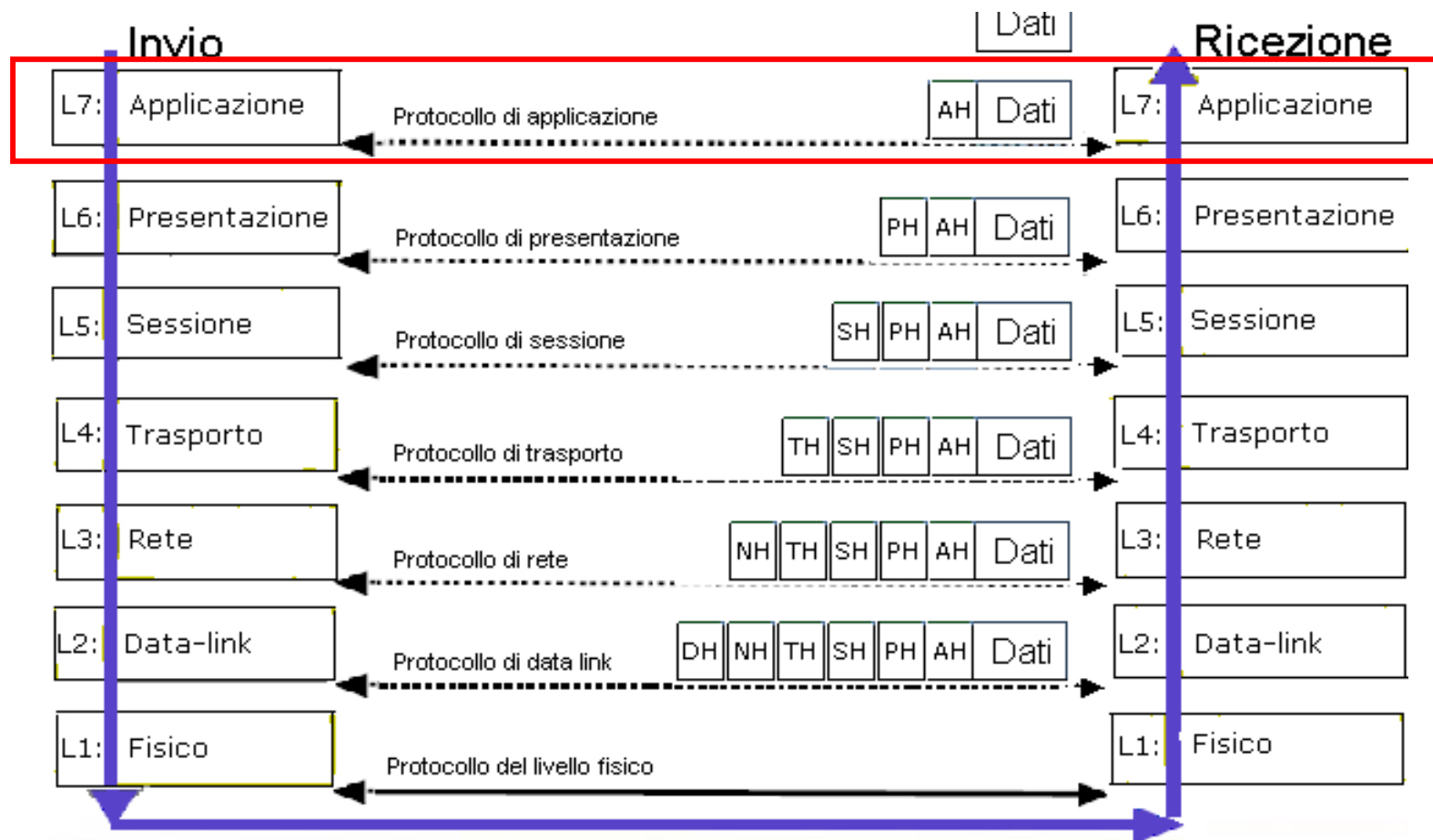
- ▶ Module II

- ▶ Analysis of application programs/protocols that take advantages from TCP/IP Internet
- ▶ Primary pattern of interaction: *client-server*

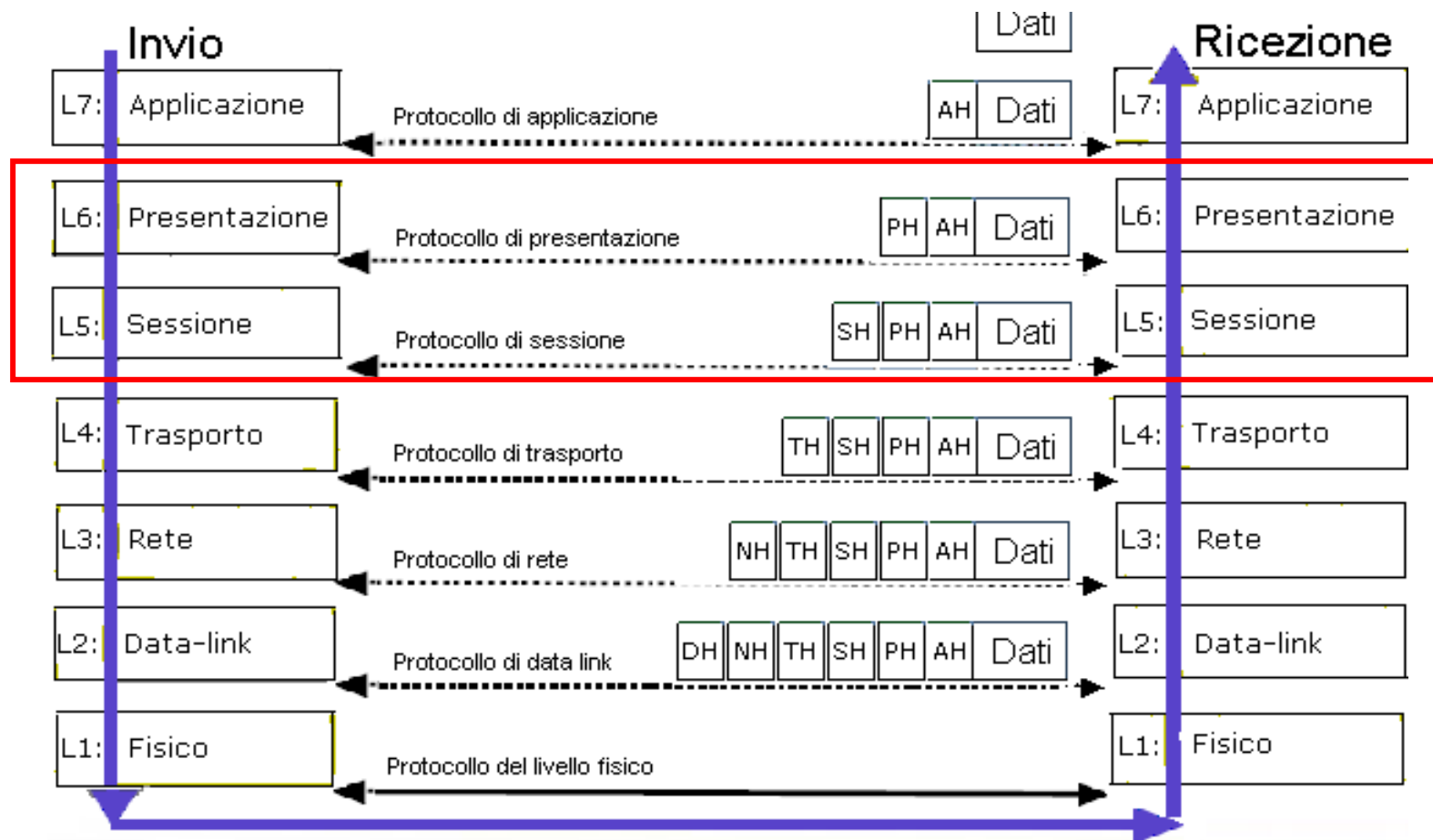
Standard ISO/OSI



Standard ISO/OSI



Standard ISO/OSI



Application Layer

- ▶ The OSI layer closest to the end user
 - ▶ Both the OSI application layer and the user interact directly with the software application
- ▶ It interacts with software applications that implement a communicating component
 - ▶ Application falls outside the scope of the OSI model
- ▶ Application layer functions typically include
 - ▶ Identifying communication partners
 - ▶ Determining resource availability
 - ▶ Synchronizing communication

Application Layer

- ▶ Identifying communication partners
 - ▶ The application layer determines the identity and availability of communication partners for an application with data to transmit
- ▶ Determining resource availability
 - ▶ The application layer must decide whether sufficient network resources for the requested communication exist
- ▶ Synchronizing communication
 - ▶ All communication between applications requires cooperation that is managed by the application layer

Application Layer Protocols

- ▶ Types of messages exchanged, e.g., request & response messages
- ▶ Syntax of message types: what fields in messages & how fields are delineated
- ▶ Semantics of the fields, i.e., meaning of information in fields
- ▶ Rules for when and how processes send & respond to messages

Application Layer Protocols

- ▶ Public-domain protocols
 - ▶ Defined in RFCs (<https://www.ietf.org/standards/rfcs/>)
 - ▶ RFC documents contain technical specifications and organizational notes for the Internet.
 - ▶ Only some RFCs are standards. Depending on their maturity level and what they cover, RFCs are labeled with different statuses: Internet Standard, Proposed Standard, Best Current Practice, Experimental, Informational, and Historic.
 - ▶ Allows for interoperability
 - ▶ For example, HTTP, SMTP
- ▶ Proprietary protocols
 - ▶ For example, KaZaA

Application Layer Protocols

- ▶ Examples of application layer protocol/implementations
 - ▶ Domain Name System (DNS)
 - ▶ Hypertext Transfer Protocol (HTTP)
 - ▶ File Transfer Protocol (FTP)
 - ▶ Simple Mail Transfer Protocol (SMTP)

Network Apps

- ▶ E-mail
- ▶ Web
- ▶ Instant messaging
- ▶ Remote login
- ▶ P2P file sharing
- ▶ Multi-user network games
- ▶ Streaming stored video clips
- ▶ Internet telephone
- ▶ Real-time video conference
- ▶ Massive parallel computing

Presentation Layer

- ▶ Provide data representation independence
 - ▶ Negotiate and select the appropriate data representation for data transfer
 - ▶ Translate from application to network format, and vice versa.
 - ▶ Transform data into a form understandable for the application layer
 - ▶ Contains functions for data security (e.g., encryption)
- ▶ Format, compress, and encrypt data to be sent across a network
 - ▶ Provide freedom from compatibility problems
 - ▶ Require a representation known to both communication parties
- ▶ Sometimes called the “syntax layer”
- ▶ Protocols: MIME, SSL, XDR

Session Layer

- ▶ Establish, manage and terminate connections between applications
 - ▶ Support two communication endpoints in the set up and synchronization of their connections
 - ▶ It is responsible for the creation and termination of a communication channel
 - ▶ Manage full-duplex and half-duplex communications
 - ▶ It deals with session and connection coordination
- ▶ Protocols: SOCKS, NetBIOS, Session establishment in TCP, RPC, NFS

Transport Layer

- ▶ It works as a trait d'union (interface) between application-oriented and network-oriented layers
- ▶ Provide a message transfer solution that is independent by the network level
- ▶ Can provide reliable transfer and ensure complete data transfer
 - ▶ Integrity check
 - ▶ End-to-end error recovery
 - ▶ Flow control (packet ordering, no packet loss, no duplicates, quality of service)
- ▶ Protocols: UDP, TCP

Transport Service

- ▶ Data loss
 - ▶ Some apps (e.g., audio) can tolerate some loss
 - ▶ Other apps (e.g., file transfer, telnet) require 100% reliable data transfer
- ▶ Timing
 - ▶ Some apps (e.g., Internet telephony, interactive games) require low delay to be “effective”
- ▶ Bandwidth
 - ▶ Some apps (e.g., multimedia) require minimum amount of bandwidth to be “effective”
 - ▶ Other apps (“elastic apps”) make use of whatever bandwidth they get

Transport Service

Application	Data loss	Bandwidth	Time Sensitive
file transfer	no loss	elastic	no
e-mail	no loss	elastic	no
Web documents	no loss	elastic	no
real-time audio/video	loss-tolerant	audio: 5kbps-1Mbps video:10kbps-5Mbps	yes, 100's msec
stored audio/video	loss-tolerant	same as above	yes, few secs
interactive games	loss-tolerant	few kbps up	yes, 100's msec
instant messaging	no loss	elastic	yes and no



Transport Service Protocols

▶ TCP service

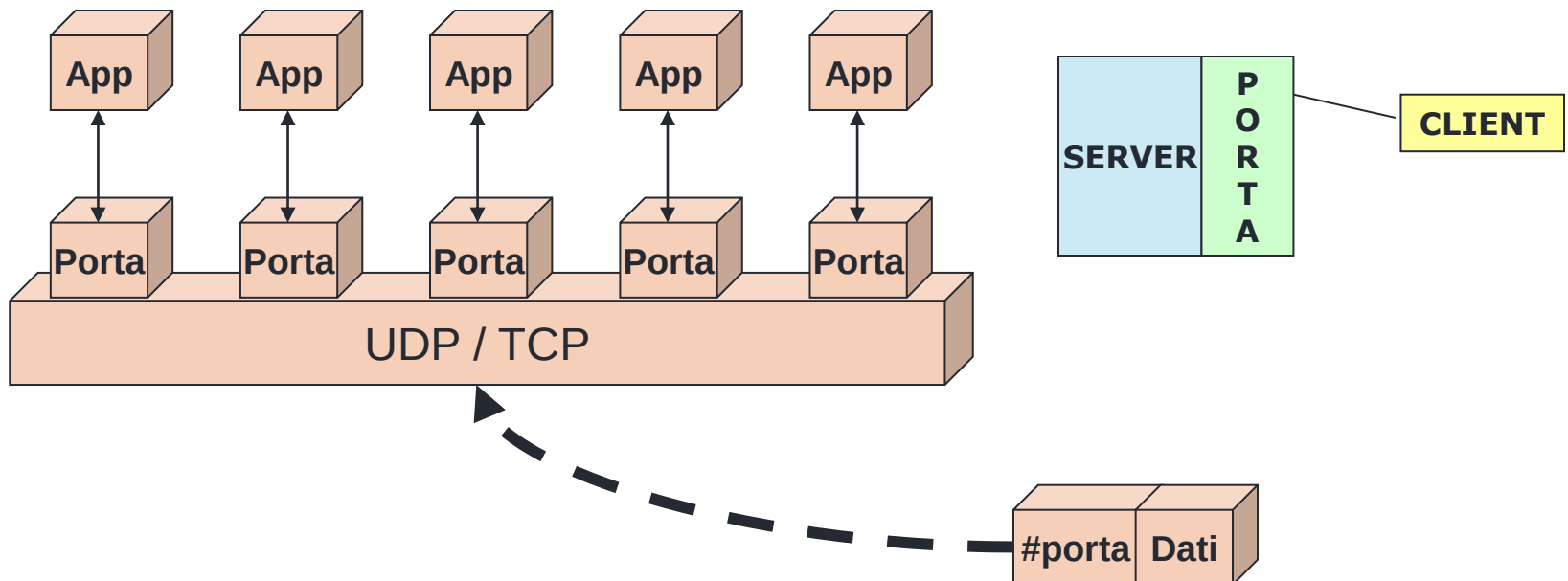
- ▶ *Connection-oriented*: setup required between client and server processes
- ▶ *Reliable transport* between sending and receiving process
- ▶ *Flow control*: sender won't overwhelm receiver
- ▶ *Congestion control*: throttle sender when network overloaded
- ▶ *Does not provide*: timing, minimum bandwidth guarantees

Transport Service Protocols

- ▶ UDP service:
 - ▶ *Unreliable data transfer* between sending and receiving process
 - ▶ *Does not provide*: connection setup, reliability, flow control, congestion control, timing, or bandwidth guarantee

Transport Service Protocols

- ▶ TCP/UDP based on the concept of **port**
 - ▶ Simple number (16-bit integer)
 - ▶ Used to map incoming data on an active process
 - ▶ Used by TCP/UDP identify the application to which data are directed
 - ▶ Well-known port reserved for specific applications



Transport Service Protocols

Application	Application layer protocol	Underlying transport protocol
e-mail	SMTP [RFC 2821]	TCP
remote terminal access	Telnet [RFC 854]	TCP
Web	HTTP [RFC 2616]	TCP
file transfer	FTP [RFC 959]	TCP
streaming multimedia	proprietary (e.g. RealNetworks)	TCP or UDP
Internet telephony	proprietary (e.g., Dialpad)	typically UDP





Client-Server: Communication Paradigm



Client-Server Paradigm

- ▶ Conceptual basis for virtually all distributed applications
- ▶ One program initiates interaction (client) to which another program responds (server)
- ▶ Note: “peer-to-peer” applications use client-server paradigm internally

Definitions

▶ Client

- ▶ Any application program
- ▶ Contacts a server
- ▶ Forms and sends a request
- ▶ Awaits a response

▶ Server

- ▶ Usually a specialized program that offers a service
- ▶ Awaits a request
- ▶ Computes an answer
- ▶ Issues a response

Server: Characteristics

- ▶ Servers usually implemented as application programs
 - ▶ Process, user process, task
 - ▶ They can be executed on system supporting TCP/IP communication
 - ▶ Timesharing systems
 - ▶ Personal computer
- ▶ Servers with same service can be executed on the same or different machines

Server Persistence

- ▶ *A server starts execution before interaction begins and (usually) continues to accept requests and send responses without ever terminating*
- ▶ *A client is any program that makes a request and awaits a response; it (usually) terminates after using a server a finite number of times*

Illustration Of The Client-Server Paradigm

- ▶ Client sends a request over the Net
 - ▶ IP address + PORT

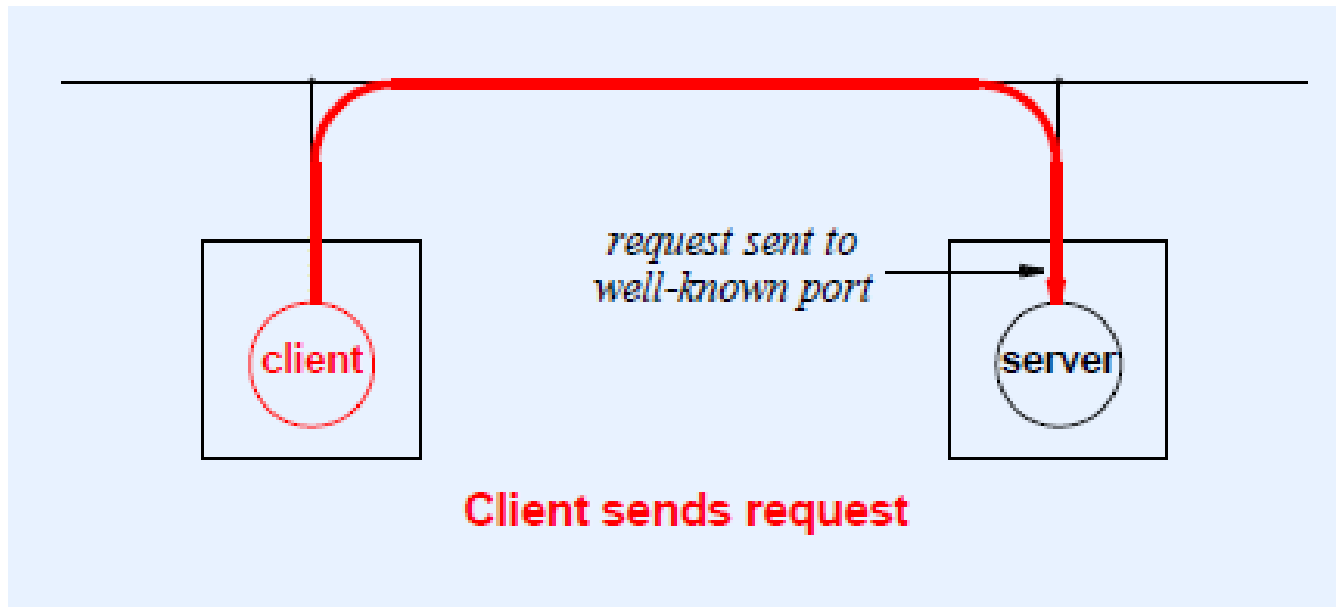
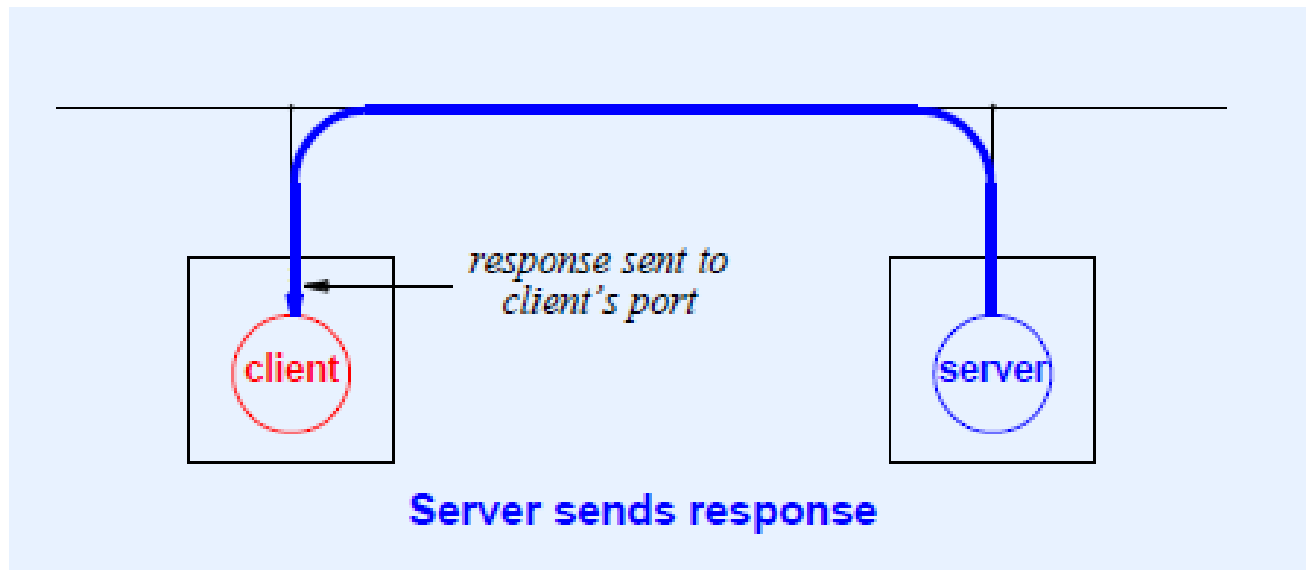


Illustration Of The Client-Server Paradigm

- ▶ Server sends a response over the Net
 - ▶ IP address + PORT



Use Of Protocol Ports

- ▶ *A server waits for requests at a well-known port that has been reserved for the service it offers*
- ▶ *A client allocates an arbitrary, unused, nonreserved port for its communication*

Example: UDP Echo Server

- ▶ Simplest form of client-server interaction
 - ▶ Unreliable datagram delivery for messages exchange between client and server
- ▶ UDP Echo Server process
 - ▶ Reserve an UDP port
 - ▶ Enter a 3-step process
 1. Wait for a datagram
 2. Reverse source and destination addresses
 3. Send back the datagram

Example: UDP Echo Client

- ▶ Allocate an unused UDP port
- ▶ Send a UDP message
- ▶ Wait for the response (the same data)

Other Examples

- ▶ Time-of-day Server
 - ▶ Very simple
 - ▶ Request and response in a single datagram
 - ▶ Representation as the number of seconds after a given epoch
- ▶ Web Server
 - ▶ Complex
 - ▶ Exchange of Web pages

Client Side

- ▶ Any application program can become a client
- ▶ Must know how to reach the server
 - ▶ Server's Internet address
 - ▶ Server's protocol port number
- ▶ Usually easy to build

Server Side

- ▶ Finds client location from incoming request
- ▶ Can be implemented within an application program or the operating system
- ▶ Starts execution before requests arrive
- ▶ Must ensure client is authorized
- ▶ Must uphold protection rules
- ▶ Must handle multiple, concurrent requests
- ▶ Usually complex to design and build

Concurrent Server Algorithm

- ▶ Our example treated sequential servers
 - ▶ Operating system **queues concurrent requests**
- ▶ Server much more difficult
 - ▶ Have to **manage concurrent requests**
- ▶ Server has two parts
 - ▶ A **master program**
 - ▶ **One or more slave programs**
 - ▶ **Slave process proceeds concurrently**

Concurrent Server Algorithm

1. Open well-known port
2. Wait for next client request
3. If necessary, create a new local protocol port for the client
4. Create thread / process to handle request (start a slave)
5. Continue with *wait* (Step 2)

Complexity of Servers

- ▶ Complexity due to concurrent requests is not the only that affect the server
- ▶ The server must enforce authorization and protection rules (e.g., access control)
- ▶ Servers execute with highest privileges (read file system, access data, keep logs...)
- ▶ Server cannot blindly honor requests from other site

Complexity of Servers

- ▶ Server (programs) must protect the operating system and its data by enforcing access and protection policies
- ▶ Also, server protects itself against malformed requests
 - ▶ File Server developed at Purdue University
 - ▶ Request to open /dev/tty cause server abort
 - ▶ Worm developed by a student at Cornell University
 - ▶ November 1988 Morris worm
 - ▶ 10% of the PC infected
 - ▶ Based on known vulnerabilities in Unix sendmail, finger, and rsh/rexec, as well as weak passwords

Complexity of Servers

- ▶ *Servers are usually more difficult to build than clients because, although they can be implemented with application programs, servers must enforce all the access and protection policies of the computer system on which they run and must protect themselves against all possible errors*

RARP Server

- ▶ Implicit assumption: client knows server address
- ▶ RARP used to allow a machine to find its IP address at startup
- ▶ RARP is an example of a different client-server interaction
 - ▶ RARP client broadcasts its request at network level rather than directly contact the server
 - ▶ RARP servers reply to the request

UDP Echo vs RARP Server

- ▶ Two main differences
 - ▶ RARP packets stored in hardware frames and sent in the **physical network**
 - ▶ UDP Echo uses IP datagrams
 - ▶ RARP cannot be implemented as an application program (needs access to hardware packets)
 - ▶ UDP Echo built as an application program

Alternatives to client-server

- ▶ ARP: IP address -> physical address
 - ▶ Modified client-server model
 - ▶ Machines keep a cache of responses
 - ▶ Improve performance and efficiency but it is still a client-server model
 - ▶ How to improve performance for the first request?
 - ▶ **Precollection** where background activities collect and propagate information before requests

Alternatives to client-server

- ▶ 4BSD UNIX ruptime command
 - ▶ Report CPU load and time since startup
 - ▶ Based on precollection
 - ▶ Clients broadcast information on the LAN
 - ▶ Clients collect information about the others
 - ▶ When needed information is already available
- ▶ Advantage
 - ▶ Speed, immediate response
 - ▶ Data available also in case of machine failure
- ▶ Disadvantage
 - ▶ Expensive in terms of processor time and network bandwidth

Summary

- ▶ Client-server model is basis for distributed applications
- ▶ Server is a specialized, complex program (process) that offers a service
- ▶ Arbitrary application can become a client by contacting a server and sending a request
- ▶ Most servers are concurrent

